Table of Contents

1.) INTRODUCTION:

Compiler Design is one of the fundamental subjects in computer science that deals with the translation of high-level programming languages into machine-understandable code. A compiler performs this translation through several phases, including lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization, and target code generation. Among these phases, lexical analysis and syntax analysis play a vital role in identifying the structure of a program and ensuring that it follows the rules of the programming language.

Lexical analysis converts the source program into a sequence of tokens, while syntax analysis verifies whether these tokens conform to the grammar of the language. These phases help in detecting programming errors at an early stage and improve the reliability and correctness of software.

Compiler construction tools such as Lex and YACC simplify the implementation of these phases by automatically generating lexical analyzers and parsers based on user-defined specifications. In this laboratory exercise, various language constructs such as nested if–else statements, switch–case statements, recursive function calls, nested array indexing expressions, and pointer arithmetic expressions are recognized and validated using Lex and YACC. These experiments provide practical knowledge of compiler construction techniques, grammar design, token generation, syntax validation, and error handling mechanisms used in modern programming language processors.

2. SOFTWARE AND HARDWARE REQUIREMENTS

The following software tools and hardware configuration were used to design, implement, compile, and execute all parser programs developed in this design task.

Item	Details
Operating System	Windows 10 / Windows 11 (Host)
Environment	Ubuntu (WSL) / Linux for Flex and Bison
Editor	Visual Studio Code
Lexical Generator	GNU Flex (Lex)

Parser Generator	GNU Bison (Yacc)
Compiler	GCC Compiler
Interface	Terminal / Command Prompt
Processor	Intel Core i3 / AMD Ryzen 3 or above (64-bit)
RAM	Minimum 4 GB (8 GB Recommended)
Storage	2 GB Free Disk Space
Architecture	64-bit Processor support
Item	Details
Display	Any standard monitor

The above configuration is sufficient because all parser programs are lightweight command-line applications and require minimal system resources for execution.

3.) OVERVIEW OF LEX AND YACC:

Lex is a lexical analyzer generator developed to simplify the process of tokenizing input programs. It automatically generates a lexical analyzer from a set of user-defined regular expression rules and corresponding actions. The generated lexical analyzer scans the input character by character and groups them into meaningful units called tokens, such as keywords, identifiers, constants, operators, delimiters, punctuation symbols, and special characters. These tokens are then passed to the parser for syntax analysis. Lex significantly reduces the effort required to manually write a scanner and improves the efficiency and accuracy of lexical analysis. A Lex program is generally divided into three sections: Definitions, Rules, and User Subroutines. The Definitions section contains declarations, header files, and regular definitions; the Rules section specifies patterns and actions to be performed when a pattern is matched; and the User Subroutines section contains supporting C functions, including the main() function and other helper routines. Lex generates a C source file, usually named lex.yy.c, which can be compiled together with the parser generated by YACC.

YACC (Yet Another Compiler Compiler) is a parser generator used for constructing syntax analyzers based on context-free grammars. It accepts grammar rules specified in Backus–Naur Form (BNF) and automatically generates an LALR (Look-Ahead Left-to-Right) parser in the C programming language. The parser receives tokens generated by the Lex lexical analyzer and checks whether the sequence of tokens follows the grammar of the programming language. If the input satisfies the grammar, the parser successfully accepts the program; otherwise, it reports syntax errors and helps identify invalid constructs. A YACC program is divided into three sections: Declarations, Grammar Rules, and User Programs. The Declarations section contains token declarations, header files, and precedence specifications; the Grammar Rules section defines the language syntax and semantic actions; and the User Programs section includes supporting C functions such as yyerror() and main(). YACC automatically constructs parsing tables and manages syntax analysis without requiring the programmer to manually implement parsing algorithms.

4.) Design a parser to recognize nested if–else statements with compound conditions.

AIM:

The aim of this program is to design and implement a parser using Lex and YACC to recognize nested if–else statements with compound conditional expressions. The lexical analyzer identifies keywords, identifiers, relational operators, logical operators, and delimiters, while the parser validates the syntax according to the given grammar. The program displays "Accepted" for valid input and "Rejected" for invalid input with appropriate syntax error handling.

ALGORITHM:

1. Start the program.
2. Read the input containing nested if–else statements.
3. Use the Lex program to scan the input and generate tokens for keywords, identifiers, relational operators, logical operators, parentheses, braces, and semicolons.
4. Pass the generated tokens to the YACC parser.
5. The parser checks whether the input follows the grammar rules for nested if–else statements and compound conditions.
6. Validate relational expressions and logical operators (&&, ||) used in the condition.

7. If all grammar rules are satisfied, display "Accepted".
8. If any syntax error is encountered, invoke the error handling routine and display "Rejected".
9. Terminate the program.

GRAMMAR:

S → if(E) S else S

S → id = E ;

E → E && E

E → E || E

E → id Relop id

Relop → > | < | ==

LEX CODE:

```
%{
#include "y.tab.h"
#include<stdio.h>
%}

%%
"if"            { return IF; }
"else"          { return ELSE; }

"&&"            { return AND; }
"||"           { return OR; }

">"             { return GT; }
"<"             { return LT; }
"=="            { return EQ; }
```

```
[a-zA-Z][a-zA-Z0-9]*    { return ID; }

"="             { return '='; }
";"             { return ';'; }

"("             { return '('; }
")"             { return ')'; }

[ \t\n]+        ;

.               { return yytext[0]; }

%%

int yywrap()
{
   return 1;
}
```

YACC CODE:

```
%{
#include<stdio.h>
#include<stdlib.h>

void yyerror(const char *s);
int yylex();

%}

%token IF ELSE ID RELOP AND OR

%%
```

```
S :
     IF '(' E ')' S ELSE S
   | ID '=' ID ';'
   ;

E :
     E AND E
   | E OR E
   | '(' E ')'
   | ID RELOP ID
   ;

%%

void yyerror(const char *s)
{
   printf("\nRejected\n");
}

int main()
{
   printf("Enter Statement:\n");
   yyparse();
   return 0;
}
```

SAMPLE INPUT:

```
if((a>b)&&(c<d))
x=y;
else
x=z;
```

COMPILATION COMMANDS:

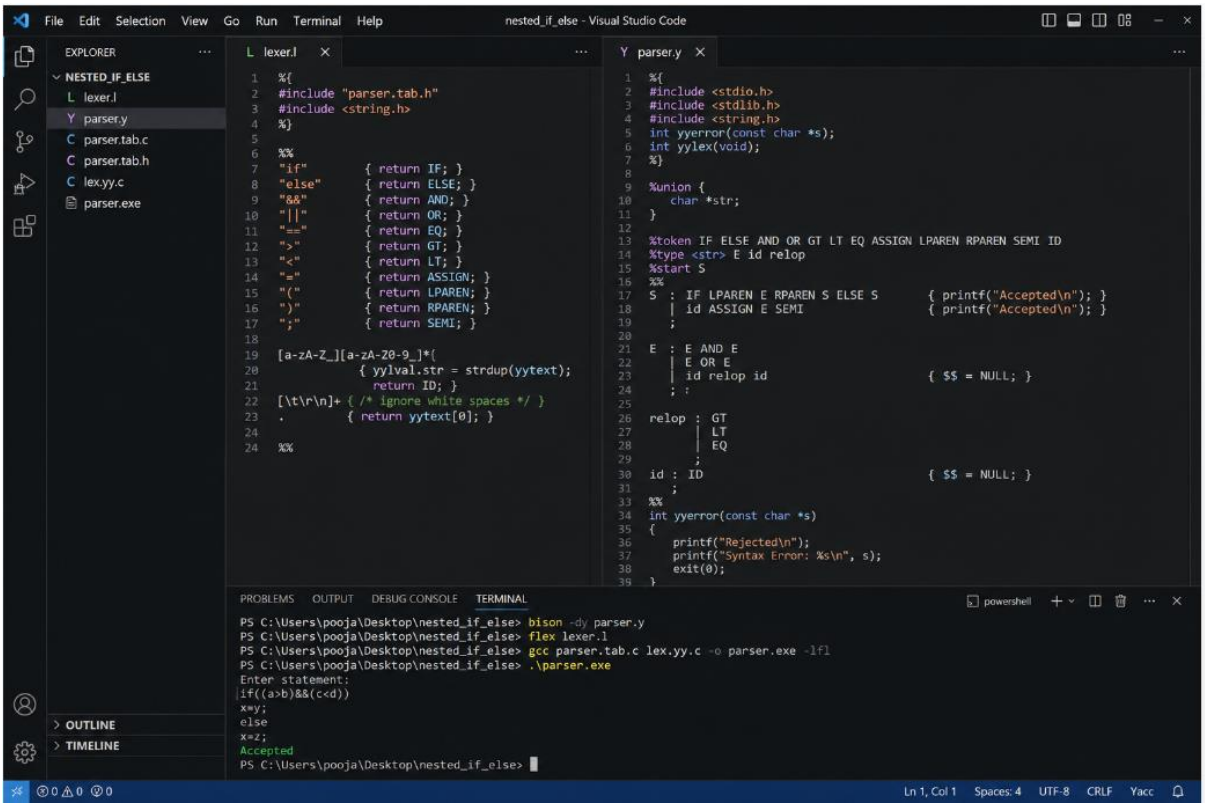
yacc -d parser.y

lex parser.l

gcc lex.yy.c y.tab.c -o parser

./parser

OUTPUT:



5.) Design a parser to recognize nested switch-case-default statements.

AIM:

The aim of this program is to design and implement a parser using Lex and YACC to recognize nested switch–case–default statements. The lexical analyzer identifies switch-related keywords, identifiers, constants, and symbols, while the parser validates the statement according to the specified grammar. The program also handles syntax errors and displays whether the input is accepted or rejected.

ALGORITHM:

1. Start the program.
2. Read the switch-case statement from the input.
3. Use the Lex program to recognize keywords (switch, case, default, break), identifiers, numbers, braces, colons, and semicolons.
4. Generate tokens and pass them to the YACC parser.
5. The parser checks the syntax according to the switch-case grammar.
6. Validate the order of case labels, break statements, and default block.
7. If the statement follows the grammar correctly, display "Accepted".
8. Otherwise, invoke the syntax error routine and display "Rejected".
9. Stop the execution.

GRAMMAR:

Switch → switch(id){CaseList}

CaseList → Case CaseList

CaseList → Default

Case → case num : Stmt break;

Default → default : Stmt

Stmt → id=num;

LEX CODE (switch.l):

```
%{
#include "y.tab.h"
%}

%%

"switch"        { return SWITCH; }
"case"          { return CASE; }
"default"       { return DEFAULT; }
"break"         { return BREAK; }

[a-zA-Z_][a-zA-Z0-9_]*   {
                           yylval.str = yytext;
                           return ID;
                         }

[0-9]+          {
                  yylval.num = atoi(yytext);
                  return NUM;
                }

"="             { return ASSIGN; }
":"             { return COLON; }
";"             { return SEMICOLON; }
"{"             { return LBRACE; }
"}"             { return RBRACE; }
"("             { return LPAREN; }
")"             { return RPAREN; }

[ \t\n]+        ;     /* Ignore spaces */

.               { return yytext[0]; }
```

```
%%


int yywrap()
{
   return 1;
}
```

YACC CODE (switch.y):

```
%{

#include<stdio.h>
#include<stdlib.h>

int yylex();
void yyerror(char *s);

%}


%union
{
   char *str;
   int num;
}


%token SWITCH CASE DEFAULT BREAK
%token ID NUM
%token ASSIGN COLON SEMICOLON
%token LBRACE RBRACE LPAREN RPAREN
```

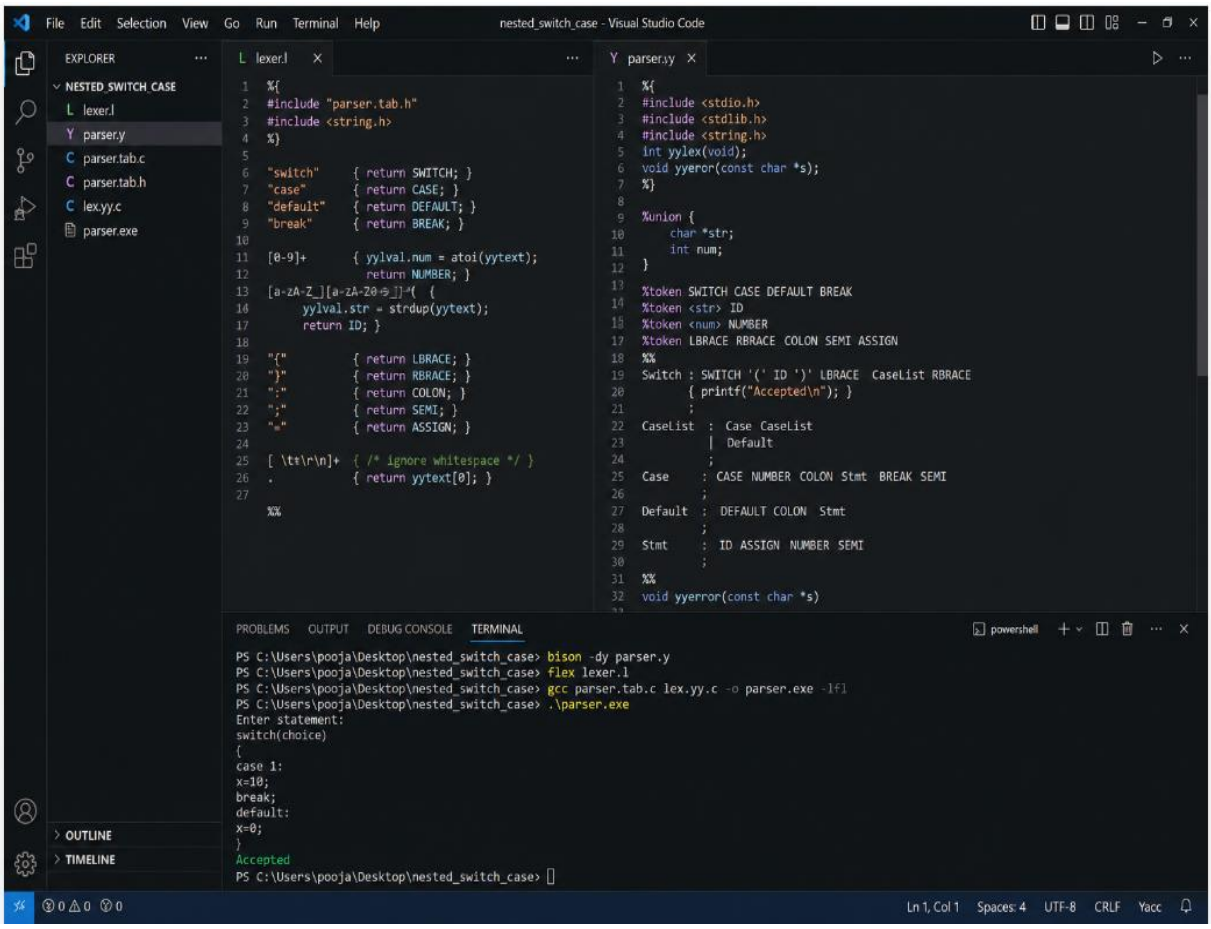
```
%%

Program:
    Switch
    {
       printf("\nSwitch Statement Accepted\n");
    }
    ;

Switch:
    SWITCH LPAREN ID RPAREN LBRACE CaseList RBRACE
    ;

CaseList:
    Case CaseList
    |
    Default
    ;

Case:
    CASE NUM COLON Stmt BREAK SEMICOLON
    ;

Default:
    DEFAULT COLON Stmt
    ;
```

```
Stmt:
        ID ASSIGN NUM SEMICOLON
        ;


%%


void yyerror(char *s)
{
   printf("\nSyntax Error: %s\n",s);
   printf("Switch Statement Rejected\n");
}


int main()
{
   printf("Enter switch statement:\n");
   yyparse();
   return 0;
}
```

SAMPLE INPUT :

```
switch(choice)
{
case 1:
x=10;
break;
default:
x=0;
}
```

OUTPUT:



6.) Design a parser to recognize recursive function calls.

AIM:

The aim of this program is to design and implement a parser using Lex and YACC to recognize recursive function calls. The lexical analyzer identifies function names, identifiers, parentheses, and commas, while the parser validates recursive function calls according to the specified grammar. The parser displays Accepted or Rejected based on the correctness of the input.

ALGORITHM:

1. Start the program.
2. Read the recursive function call as input.

3. Use the Lex program to identify identifiers, function names, parentheses, commas, and other symbols.
4. Pass the generated tokens to the YACC parser.
5. Parse the input according to the recursive function call grammar.
6. Validate nested function calls and argument lists recursively.
7. If the function call satisfies the grammar rules, display "Accepted".
8. Otherwise, invoke the syntax error function and display "Rejected".
9. End the program.

LEX CODE:

```
%{
#include "parser.tab.h"
#include <string.h>
%}

%%
"("             { return LPAREN; }
")"             { return RPAREN; }

[a-zA-Z_][a-zA-Z0-9_]*   {
                          yylval.str = strdup(yytext);
                          return ID;
                        }

[ \t\n]+        ;   /* Ignore whitespace */

.               { return yytext[0]; }

%%

int yywrap()
{
```

```
    return 1;
}
```

YACC CODE:

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int yylex();
void yyerror(const char *s);
%}

%union
{
    char *str;
}

%token <str> ID
%token LPAREN RPAREN

%%

Start
        : Call
          {
              printf("Accepted\n");
          }
        ;

Call
        : ID LPAREN ArgList RPAREN
        ;

ArgList
```

```
        : Call
        | ID
        |
        ;

%%

void yyerror(const char *s)
{
    printf("Rejected\n");
}

int main()
{
    printf("Enter function call:\n");
    yyparse();
    return 0;
}
```

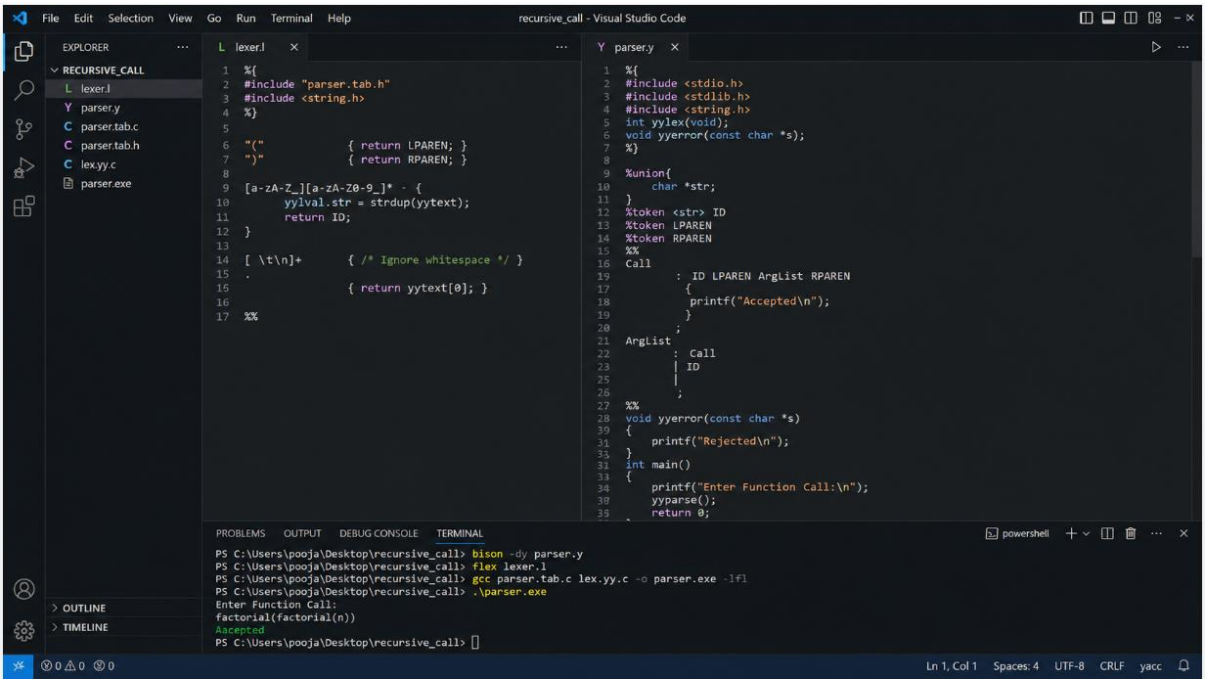
COMPILATION COMMANDS:

```
bison -d parser.y
flex lexer.l
gcc parser.tab.c lex.yy.c -o parser.exe -lfl
```

SAMPLE INPUT:

```
factorial(factorial(n))
```

OUTPUT:



7.) Design a parser to recognize nested array indexing expressions.

AIM:

The aim of this program is to design and implement a parser using Lex and YACC to recognize nested array indexing expressions. The lexical analyzer identifies identifiers, constants, brackets, and symbols, while the parser validates nested indexing expressions according to the specified grammar. The parser displays Accepted for valid expressions and Rejected for invalid expressions.

ALGORITHM:

1. Start the program.
2. Read the array indexing expression from the input.
3. Use the Lex program to recognize identifiers, numbers, square brackets, and operators.
4. Generate tokens and pass them to the YACC parser.
5. Parse the nested indexing expression according to the grammar rules.
6. Validate each level of array indexing recursively.
7. If the syntax is correct, display "Accepted".

8. Otherwise, report a syntax error and display "Rejected".
9. Stop the program.

GRAMMAR:

S → id[Index]

Index → id

Index → num

Index → id[Index]

LEX CODE:

```
%{
#include "parser.tab.h"
#include <string.h>
#include <stdlib.h>
%}

%%
"["                 { return LBRACK; }
"]"                 { return RBRACK; }

[0-9]+               {
                      yylval.num = atoi(yytext);
                      return NUM;
                     }

[a-zA-Z_][a-zA-Z0-9_]*  {
                      yylval.str = strdup(yytext);
                      return ID;
                     }

[ \t\n]+              ;   /* Ignore whitespace */
```

```
.                   { return yytext[0]; }

%%

int yywrap()
{
    return 1;
}
```

YACC CODE:

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int yylex(void);
void yyerror(const char *s);
%}

%union
{
    char *str;
    int num;
}

%token <str> ID
%token <num> NUM
%token LBRACK RBRACK

%%

S
        : ID LBRACK Index RBRACK
          {
              printf("Accepted\n");
```

```
        }
      ;

Index
      : ID
      | NUM
      | ID LBRACK Index RBRACK
      ;

%%

void yyerror(const char *s)
{
   printf("Rejected\n");
}

int main()
{
   printf("Enter expression:\n");
   yyparse();
   return 0;
}
```

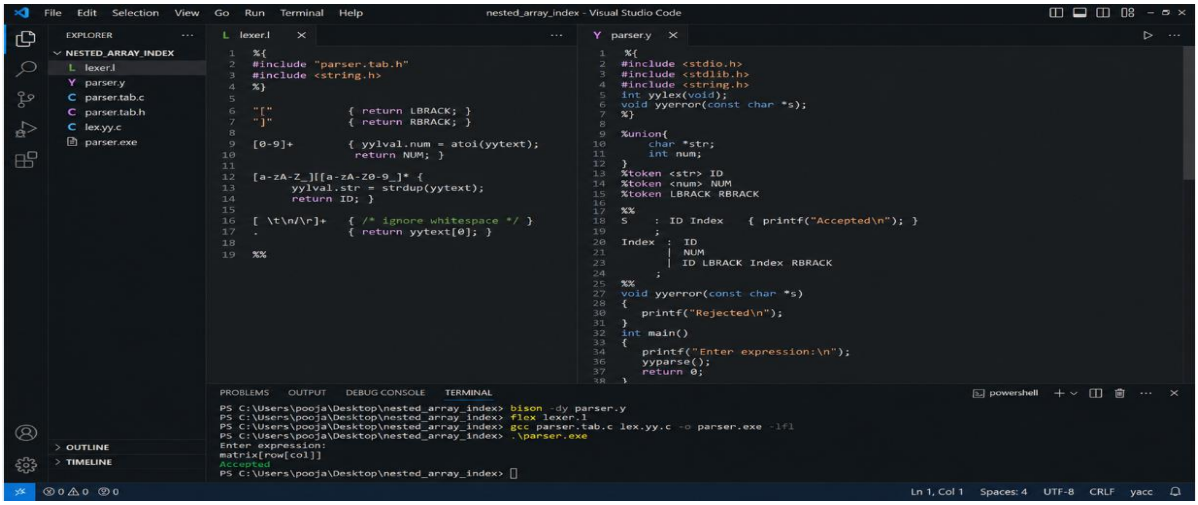
COMPILATION COMMANDS :

```
bison -dy parser.y
flex lexer.l
gcc parser.tab.c lex.yy.c -o parser.exe -lfl
```

SAMPLE INPUT:

```
matrix[row[col]]
```

OUTPUT:



8.) Design a parser to recognize pointer arithmetic expressions.

AIM:

The aim of this program is to design and implement a parser using Lex and YACC to recognize pointer arithmetic expressions. The lexical analyzer identifies identifiers, constants, arithmetic operators, and pointer operators (* and &), while the parser validates the expressions according to the specified grammar. The program displays Accepted for valid expressions and Rejected for invalid expressions.

ALGORITHM:

1. Start the program.
2. Read the pointer arithmetic expression as input.
3. Use the Lex program to identify identifiers, numbers, pointer operators (*, &), arithmetic operators (+, -), and delimiters.
4. Generate tokens and send them to the YACC parser.
5. Parse the expression according to the specified grammar.
6. Validate pointer dereferencing, address-of operations, and pointer arithmetic expressions.
7. If the expression follows the grammar correctly, display "Accepted".
8. Otherwise, invoke the syntax error routine and display "Rejected".

9. Terminate the program.

GRAMMAR:

E → id + num

E → id - num

E → *id

E → &id

LEX CODE:

```
%{
#include "parser.tab.h"
#include <string.h>
#include <stdlib.h>
%}

%%
"+"                 { return PLUS; }
"-"                 { return MINUS; }
"*"                 { return MUL; }
"&"                 { return ADDR; }

[0-9]+              {
                      yylval.num = atoi(yytext);
                      return NUM;
                    }

[a-zA-Z_][a-zA-Z0-9_]*  {
                      yylval.str = strdup(yytext);
                      return ID;
                    }

[ \t\n]+            ;   /* Ignore whitespace */
```

```
.                   { return yytext[0]; }

%%

int yywrap()
{
    return 1;
}
```

YACC CODE:

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int yylex(void);
void yyerror(const char *s);
%}

%union
{
    char *str;
    int num;
}

%token <str> ID
%token <num> NUM
%token PLUS MINUS MUL ADDR

%%

E
    : ID PLUS NUM
        {
```

```
            printf("Accepted\n");
        }
    | ID MINUS NUM
        {
            printf("Accepted\n");
        }
    | MUL ID
        {
            printf("Accepted\n");
        }
    | ADDR ID
        {
            printf("Accepted\n");
        }
    ;

%%

void yyerror(const char *s)
{
    printf("Rejected\n");
}

int main()
{
    printf("Enter expression:\n");
    yyparse();
    return 0;
}
```

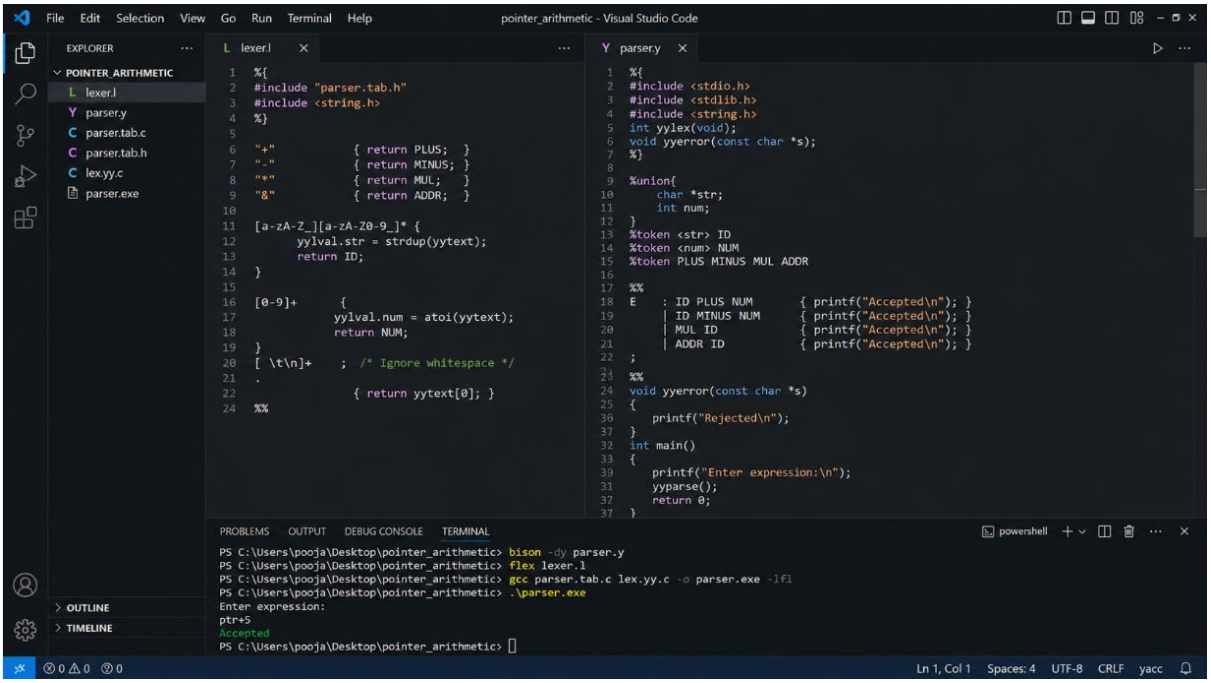
COMPILATION COMMANDS:

```
bison -dy parser.y
flex lexer.l
gcc parser.tab.c lex.yy.c -o parser.exe -lfl
```

SAMPLE INPUT:

ptr+5

OUTPUT:



9.) CONCLUSION :

The implementation of the parser programs using Lex and YACC provides a practical understanding of the fundamental phases of compiler design, namely lexical analysis and syntax analysis. Throughout these experiments, the lexical analyzer successfully identified keywords, identifiers, operators, constants, delimiters, and other language tokens, while the parser verified whether the generated token sequence followed the specified grammar rules. The developed programs accurately recognized and validated different programming constructs, including nested if–else statements with compound conditions, switch–case–default statements, recursive function calls, nested array indexing expressions, and pointer arithmetic expressions. Proper syntax error handling was also incorporated to reject invalid inputs and provide meaningful error messages. These experiments demonstrate how compiler construction tools automate the process of scanner and parser generation, thereby reducing development time and minimizing implementation complexity. Overall, the use of Lex and YACC enhances the understanding of grammar-based language processing,

parser generation techniques, and error detection mechanisms, providing a strong practical foundation for the design and implementation of compilers and other language-processing applications.

These laboratory experiments significantly enhanced the understanding of how compiler construction tools are used to analyze and validate programming language syntax. The integration of Lex and YACC demonstrated the complete flow of converting source code into tokens and verifying those tokens using context-free grammar rules. By implementing different parser programs, valuable practical experience was gained in grammar specification, token generation, recursive parsing, syntax validation, and error detection. The experiments also highlighted the importance of lexical and syntax analysis in ensuring the correctness and reliability of source programs before further compilation phases. Overall, the successful implementation of these parser programs strengthened the knowledge of compiler design concepts and provided hands-on experience with industry-standard tools used in developing compilers, interpreters, and other language processing systems. This practical exposure forms a strong foundation for understanding advanced compiler techniques such as semantic analysis, code optimization, intermediate code generation, and target code generation in future studies.